Foreword

This guide was written to help people who aren't coders learn how to make scripts that bring their characters and worlds to life. For many, scripting feels like a mysterious wall of symbols and rules. My goal is to show you that it's not magic — it's just building blocks stacked carefully, one step at a time.

Whether you're here to build roleplay characters, manage world lore, or just tinker for fun, I want this guide to be something you can flip through without feeling lost.

Remember: scripting is not about writing the "perfect" code. It's about creating something that feels alive, fun, and responsive. Start small, experiment, and let your characters grow with you.

— *Icehellionx*

Introduction

Welcome to the **Script Making Guide**, a beginner-to-advanced handbook for writing scripts in a sandbox environment.

This book starts at the absolute basics:

- What a script is.
- How it interacts with personality and scenario.
- Simple keyword checks.

Then it builds steadily into intermediate and advanced topics:

- Lore structures.
- Probability and randomization.
- Message count gating.
- Reaction engines.
- Emotional hybrids.
- Modular frameworks like the **Everything Lorebook.**

By the time you finish, you'll know how to:

- Write safe scripts that won't crash.
- Make conversations feel dynamic and immersive.
- Organize large worlds into neat, reusable modules.

The only requirement? Curiosity. If you can read and follow examples step-by-step, you can script.

This guide has been written by **Icehellionx**, and shaped through weeks of testing, trial, and improvement. Every example, every snippet, has been verified to work in the sandbox environment described.

Think of this book as your roadmap: start at Chapter 1, take it one step at a time, and by Chapter 25 you'll be building full lore engines of your own.

Chapter List

Reference

Chapter 1: What Scripts Are

Imagine your character is like an actor in a play. Normally, they follow the script you wrote in their **Personality** and **Scenario** fields. But what if you wanted them to change their lines on the fly depending on what the user says? That's where **scripts** come in.

A **script** is just a tiny set of instructions you write in a very basic form of JavaScript. Think of it like a recipe card:

- If this happens → do that.
- If the user says "hello" → make the character smile.

When Do Scripts Run?

Scripts are automatic. You don't press a button to activate them. They run:

- Before every single bot reply.
- Right after the user sends a message.
- Every time the chat moves forward.

That means your script is always "listening," watching what the user says, and adjusting the character's behavior in the background.

What Can Scripts Change?

This is important: scripts only get to change **two things**:

1. **Personality** → how the character acts or feels
    - Example: "cheerful and supportive"

2. **Scenario** → what's happening around the character
    - Example: "It's late at night, and the rain is tapping on the window."

Everything else (like the character's name, memories, or chat history) is locked. The sandbox doesn't let you mess with it. Think of **Personality** and **Scenario** as two whiteboards you're allowed to write on.

The Context Object (Your Toolbox)

When your script runs, it's given a **context object**. This is just a box of information about the current chat.

- `context.character` → everything about your character
- `context.chat` → information about the chat itself

You'll use these most (written on separate lines so you can copy them easily):

```
context.character.personality // you can add personality traits here
context.character.scenario // you can add scene details here
context.chat.last_message // the last thing the user typed
context.chat.message_count // how many messages have been sent so far
```

Plain English:

- personality is the actor's mood.
- scenario is the stage set.
- last_message is the latest line from the audience.
- message_count is how many lines have been spoken so far in the play.

A Tiny First Example

Example: simple "hello" trigger

```
if (context.chat.last_message.toLowerCase().indexOf("hello") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
}
```

What it means:

- Look at what the user just typed.
- Change it to lowercase so "HELLO" or "Hello" also works.

- Check if the word "hello" is in there.
- If yes, add two short notes: one to scenario (what's happening) and one to personality (how they act).

Key Takeaways from Chapter 1

- Scripts are *if-this-then-that* instructions for your characters.
- They only control **Personality** and **Scenario**.
- They run **every time the user types something**.
- You use the `context` toolbox to get info about the conversation.
- With just a few lines, you can make your character react to words.

Chapter 2: The Context Object (Your Toolbox)

When your script runs, it doesn't start from scratch — it's handed a **context object**.
Think of the **context** like a backpack your script always carries around. Inside, you'll find everything you need to know about:

- **The character** (their personality, scenario, etc.)
- **The chat** (what the user just said, how many messages have happened)

Inside the Character Backpack

The script gives you a special box called `context.character`.

Here are the items inside (some you can use, some you can't):

- `name` → the character's actual name (read-only)
- `chat_name` → the nickname shown in chat (read-only)
- `example_dialogs` → example lines (read-only)
- `personality` → **the character's mood and traits (YOU can change this)**
- `scenario` → **the current situation or setting (YOU can change this)**

Plain English:

- `name` is their **driver's license** — you can read it but not change it.

- `chat_name` is their **nametag in the play** — fixed.
- `example_dialogs` are their **practice lines** — not for editing.
- `personality` is the **actor's mood** — you can rewrite it mid-play.
- `scenario` is the **stage set** — you can rearrange it.

Inside the Chat Backpack

The script also gives you `context.chat`. This holds details about the conversation itself:

- `message_count` → how many messages have been sent so far
- `last_message` → the most recent thing the user typed
- `first_message_date` → when the conversation began (optional)
- `last_bot_message_date` → when the bot last replied (optional)

Plain English:

- `message_count` is like the **line number** in the play.
- `last_message` is the **last thing shouted from the audience**.
- `first_message_date` and `last_bot_message_date` are like **timestamps** — nice extras, but most beginners won't need them.

Why Only Two Things Can Change

Remember: in scripts, you are only allowed to modify:

- `context.character.personality`
- `context.character.scenario`

Everything else is locked.
Why? Because it keeps the sandbox safe. If scripts could change names, rewrite chat logs, or delete stuff, everything could break.

Think of it like this: You're allowed to **write on the whiteboards (personality & scenario)**, but you can't tear down the walls.

Example: Using the Context

Example: printing context info for testing

```
console.log("Last message was:", context.chat.last_message);
console.log("Total messages so far:", context.chat.message_count);
console.log("Character's current personality:", context.character.personality);
```

What this does:

- Prints the last thing the user said.
- Prints how long the conversation has gone.
- Prints the personality text so you can see what's been added already.

These logs are only for testing — they don't show up in normal chat, just in the debug panel.

A First Real Example

Example: warming up after a long chat

```
if (context.chat.message_count > 20) {
context.character.personality += ", has really warmed up to the user";
context.character.scenario += " The atmosphere feels much friendlier now.";
}
```

Plain English:

- If the chat has gone past 20 messages...
- Add to **personality**: "has really warmed up to the user."
- Add to **scenario**: "The atmosphere feels much friendlier now."

Result: the character feels more connected the longer you talk.

Key Takeaways from Chapter 2

- The **context object** is your script's toolbox.
- `context.character` → info about the character (you only edit *personality* and *scenario*).
- `context.chat` → info about the conversation (you can read things like the last user message or message count).
- Scripts use these to make smart decisions, like reacting to certain words or changing behavior over time.

Chapter 3: The Sandbox Rules (What Works and What Doesn't)

If you've ever played a video game in **sandbox mode**, you know you can experiment — but there are still boundaries. That's exactly what happens here.

Your script doesn't run in "real" JavaScript like a professional programmer uses. Instead, it runs in a **safe sandbox** — a stripped-down version of JavaScript called **ES5** (an older edition). This sandbox keeps things simple and prevents you from accidentally breaking the system.

The Golden Rule

You can only use the tools the sandbox gives you.
If you try to use something too modern, too heavy, or unsafe, the script will fail silently (which means: nothing happens, and you'll be confused).

Safe Tools (These Always Work)

Here are the "safe toys" in your sandbox:

- **Strings (text tools):**
  - `.toLowerCase()` → makes text lowercase
  - `.indexOf("word")` → checks if a word is in a sentence
  - `.trim()` → removes spaces at the start/end
- **Numbers & Math:**
  - `+`, `-`, `*`, `/` (basic math)
  - `Math.random()` (get a random number)
  - `Math.floor()` (round a number down)
- **Arrays (lists):**
  - `arr.length` (how many items)
  - `arr.indexOf("thing")` (find a specific item)
  - Loops:

```
for (var i = 0; i < arr.length; i++){
// do something with arr[i]
}
```

- **Dates:**
  - `new Date()` (what time it is)
  - `getHours()`, `getMinutes()` (current time pieces)
- **Regular Expressions (regex):**
  - Basic keyword checks, like

```
if (/\bhello\b/i.test(text)){
// match
}
```

- Keep it simple — advanced regex (like lookbehind) doesn't work
- **Objects:**
  - `Object.keys()`, `Object.assign()` (basic object tools)
- **Debugging:**
  - `console.log()` (print something to the debug panel)

These are your bread and butter. They will *always* work.

Unsafe Tools (These Never Work)

If you try these, your script will silently fail:

- Modern JavaScript shortcuts:
  - Arrow functions `() => { ... }`
  - Template strings `Hello ${name}`
  - Spread operators `...myArray`
  - `async/await`, promises
  - Classes
- Array helpers:
  - `.map()`, `.filter()`, `.reduce()`, `.forEach()`, `.sort()`
  - Basically: anything that feels like "fancy looping"

- Object helpers:
  - `.entries()`, `.values()`, `.fromEntries()`
- Error handling:
  - `try { ... } catch { ... }`
- Timers & external stuff:
  - `setTimeout`, `setInterval`
  - Fetching data (`fetch`, `XMLHttpRequest`)
  - DOM manipulation (`document.getElementById`)

If you've seen these in modern tutorials, forget them here.

Gray Area Tools (Sometimes Work, Sometimes Don't)

Some features exist in *some* hosts, but not all. They may work one day, then fail on another platform. Use them at your own risk:

- `.includes()` → sometimes works like `.indexOf()`, sometimes doesn't
- `.padStart()`, `.padEnd()`
- `.repeat()`

Beginner tip: Don't rely on these. Stick to `.indexOf()` for word checks. It's always safe.

Why So Many Limits?

The sandbox is designed to:

1. Keep things fast → scripts run every single message, so they need to be lightweight
2. Keep things safe → you can't "hack" the system or overload it
3. Keep things simple → beginners don't need 1,000 JavaScript features

Think of it like learning to drive in a **practice car with a speed limiter**. You can't break the rules, but you'll learn the basics safely.

Example: Safe vs Unsafe

Unsafe (will fail):

```
if (context.chat.last_message.includes("hello")) {
context.character.scenario += "They greet you warmly.";
}
```

Safe (will work everywhere):

```
if (context.chat.last_message.toLowerCase().indexOf("hello") !== -1) {
context.character.scenario += "They greet you warmly.";
}
```

What changed?

- Replaced `.includes()` (iffy) with `.indexOf(...) !== -1` (safe)
- Lowercased the message first so it catches "Hello" or "HELLO"

Key Takeaways from Chapter 3

- You're in a **sandbox**, not full JavaScript
- Safe: text basics, math, arrays with loops, dates, regex, console.log
- Unsafe: modern features, array helpers, async, external APIs
- Gray area: `.includes`, `.padStart`, `.repeat` — avoid them if possible
- Always pick the **safest, simplest tool** when in doubt

Chapter 4: How to Match Words Safely

So far we've learned *what scripts are* (little recipes), *what the context object is* (your backpack of tools), and *what the sandbox allows* (only the safe toys). Now it's time to actually **react to words** the user types.

This is the "hello world" of scripting:

If the user says X, then make the character do Y.

Why Matching Needs to Be Careful

At first glance, you might think:

> "Just check if the user's message contains the word!"

But computers are picky. Look at this:

- User types: **“Hello there!”**
- If we only check for lowercase **“hello”**, we’ll miss it.
- If we check for **“hell”**, we might accidentally match **“shells.”**

That’s why we need **safe matching**.

Step 1: Normalize the Message

First, we make the user’s message lowercase so it doesn’t matter if they type “HELLO” or “hello.”

var last = context.chat.last_message.toLowerCase();

Plain English:
“Take the user’s last line and make everything lowercase.”

Step 2: Pad with Spaces

Next, we add a space at the start and end:

var padded = " " + last + " ";

Why?
So we only catch whole words.

- " hello " → matches.
- "shellows" → won’t match, because it doesn’t have spaces around it.

Step 3: Check for the Word

Now we use the safest tool: `.indexOf()` .

if (padded.indexOf(" hello ") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
}

Line by line:

- `if ( ... !== -1)` → means “if the word is found.”

- If found:
    - Add to the **scenario**: “They greet you warmly.”
    - Add to the **personality**: “Friendly and welcoming.”

Example in Action

- User types: “HELLO!!!”
- Script lowercases → “hello!!!”
- Script pads → " hello!!! "
- `indexOf(" hello ")` → finds it inside.
- Result: Character smiles and greets you.

Step 4: Match Multiple Words

What if you want to catch **hi, hey, hello** all at once?
We can use a simple list:

```
var greetings = ["hi", "hello", "hey"];
for (var i = 0; i < greetings.length; i++) {
if (padded.indexOf(" " + greetings[i] + " ") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
break; // stop after the first match
}
}
```

Plain English:

- Make a list of possible greetings.
- Loop through them one by one.
- If one matches → trigger the response.
- `break;` makes sure we don’t fire multiple times.

Step 5: Match Phrases (Two or More Words)

Sometimes you want to catch phrases like **“calm down.”** That works too:

```
if (padded.indexOf(" calm down ") !== -1) {
context.character.personality += "Tries to stay calm.";
}
```

Notice: same trick, just with two words inside the quotes.

Extra Safety: Regex (Optional)

If you're feeling adventurous, you can use **regex** for trickier matches:

```
if (/\b(help|assist|aid)\b/i.test(last)) {
context.character.personality += "Eager to be helpful.";
}
```

What this means:

- `\b` = word boundary (keeps it from matching inside longer words).
- `(help|assist|aid)` = any of these words.
- `i` = ignore capitalization.

Beginner tip: Regex is powerful, but can be confusing. Stick to `.indexOf` until you're confident.

Quick Practice (Try It Yourself!)

1. Write a script that makes the character **sad** if the user types "sorry."
2. Write a script that makes the character **excited** if the user says "let's go!"
3. Write a script that makes the character **mysterious** if the user mentions "secret."

(Hint: Use `indexOf(" sorry ")`, `indexOf(" let's go ")`, etc.)

Key Takeaways from Chapter 4

- Always lowercase the message (`.toLowerCase()`).
- Always pad with spaces (`" " + last + " "`).
- Use `.indexOf(" word ") !== -1` for safe checks.
- You can catch multiple words with loops or regex.
- Keep responses short and simple.

Pro Tip: Don't try to catch *every* word at once. Start small — one or two triggers — then build up. Scripts are like Lego bricks: stack them slowly and test as you go.

Chapter 5: Progressive Examples (From Tiny Triggers to Lorebooks)

By now you've learned the basics:

- Scripts run automatically
- They only change *personality* and *scenario*
- You can safely match words and phrases

Now let's stack those building blocks into **progressive examples** — each one more advanced than the last. Think of it like leveling up in a video game: start at Level 1 (simple trigger) and end at Level 10 (dynamic lorebook).

The Progression Roadmap

- **Level 1** → Tiny Trigger (one word → one response)
- **Level 2** → Multiple Keywords
- **Level 3** → Emotion Detection
- **Level 4** → Message Count Progression
- **Level 5** → Simple Lorebook (priorities)
- **Level 6** → Scenario Lorebook (personality + scene)
- **Level 7** → Dynamic Lorebook (plain if checks)
- **Level 8** → Timed Lore Reveals (gating)
- **Level 9** → Hybrid Systems (mixing moods, hobbies, triggers)
- **Level 10** → Advanced Lorebooks (multi-pass, probabilities, unlocks)

Level 1: The Tiny Trigger

The simplest script: look for one word, react once.

```
var last = context.chat.last_message.toLowerCase();
var padded = " " + last + " ";
```

```
if (padded.indexOf(" hello ") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
}
```

Plain English:
If the user says “hello” → the bot greets them warmly.

Level 2: Multiple Keywords

Catch “hi,” “hello,” and “hey” all at once.

```
var greetings = ["hi", "hello", "hey"];
for (var i = 0; i < greetings.length; i++) {
if (padded.indexOf(" " + greetings[i] + " ") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
break; // stop after first match
}
}
```

Plain English:
Put greetings into a list, check them one by one, stop after the first match.

Level 3: Emotion Detection

Detect mood words.

```
var emotions = ["happy", "sad", "angry", "excited"];
for (var i = 0; i < emotions.length; i++) {
if (padded.indexOf(" " + emotions[i] + " ") !== -1) {
context.character.scenario += "The user seems " + emotions[i] + ".";
break;
}
}
```

Plain English:
If the user says “sad,” scenario gets: *“The user seems sad.”*

Level 4: Message Count Progression

The character changes the longer you chat.

```
var count = context.chat.message_count;

if (count < 5) {
context.character.personality += ", polite and formal";
} else if (count < 15) {
context.character.personality += ", warming up and more casual";
} else if (count < 30) {
context.character.personality += ", friendly and open";
} else {
context.character.personality += ", deeply connected and trusting";
}
```

Plain English:
Early = polite, mid = casual, later = trusting.

Level 5: Simple Lorebook

Lore entries with keywords + priorities.

```
var lorebook = [
{ keywords: ["godfather", "damien"], priority: 10, personality: ", a calculating and charismatic leader" },
{ keywords: ["mafia", "family"], priority: 5, personality: ", part of a powerful crime family" }
];

var activated = [ ];
for (var i = 0; i < lorebook.length; i++) {
var entry = lorebook[i];
for (var j = 0; j < entry.keywords.length; j++) {
if (padded.indexOf(" " + entry.keywords[j] + " ") !== -1) {
activated.push(entry);
break;
}
}
}

activated.sort(function(a, b) { return b.priority - a.priority; });
if (activated.length > 0) {
```

```
context.character.personality += activated[0].personality;
}
```

Plain English:
If multiple entries match, the one with the highest priority wins.

Level 6: Scenario Lorebook

Same as above, but adds to the scene.

```
var lorebook = [
{ keywords: ["godfather"], personality: ", calculating and powerful", scenario: "The Godfather is in a tense meeting." },
{ keywords: ["family"], personality: ", loyal to family above all", scenario: "The mafia family spreads through the city." }
];
```

Plain English:
Lore entries can expand *both* personality and scenario.

Level 7: Dynamic Lorebook (Procedural)

Plain if checks instead of arrays.

```
if (padded.indexOf(" magic ") !== -1) {
context.character.personality += ", knowledgeable about magic";
context.character.scenario += "They sense magical energies around them.";
}
```

Plain English:
Direct checks = less structure, easier for beginners.

Level 8: Timed Lore Reveals

Lore gated behind message counts.

```
if (count > 15 && padded.indexOf(" secret ") !== -1) {
context.character.personality += ", keeper of ancient secrets";
```

```
context.character.scenario += "They whisper about the Sundering.";
}
```

Plain English:
Secrets only unlock after 15+ messages.

Level 9: Hybrid Systems

Mix keywords, moods, and timing together.

```
if (padded.indexOf(" painting ") !== -1 && padded.indexOf(" happy ") !== -1) {
context.character.scenario += "They joyfully describe their love of painting.";
}

if (padded.indexOf(" forest ") !== -1 && count > 20) {
context.character.scenario += "The forest feels darker now, full of secrets.";
}
```

Plain English:

- If hobby + emotion appear together → special lore fires.
- If a place is mentioned later in chat → tone changes.

Level 10: Advanced Lorebooks (Multi-Pass)

Advanced systems can include:

- **Priorities** (important traits win)
- **Probabilities** (some fire only half the time)
- **Unlocks** (one entry unlocks another)

Plain English:
This is where characters feel truly "alive," combining memory, timing, emotion, and probability.

Recap Table

Level	What It Adds	Example
1	One word trigger	"hello" → greets you
3	Emotion detection	"sad" → "user seems sad"

Level	What It Adds	Example
4	Message progression	Early polite → later trusting
5	Lorebook with priority	Godfather beats mafia
6	Lorebook with scenario	Expands scene too
8	Timed reveals	Secrets unlock after 15 lines
9	Hybrid logic	Hobby + emotion = special
10	Advanced multi-pass	Priorities + probabilities + unlock chains

---

Key Takeaways from Chapter 5

- Start tiny (Level 1)
- Add complexity step by step
- Use **priorities** to resolve conflicts
- Gate lore with **timing** for progression
- Hybrid/advanced lorebooks create evolving, lifelike characters

---

Pro Tip: You don't need to rush to Level 10. The sweet spot for beginners is Levels 3–6 (emotion detection, message progression, and simple lorebooks). That's where you'll have the most fun without headaches.

Chapter 6: Dynamic Behaviors (Time, Message Count, and Events)

So far we've built scripts that react to **words**. But what if you want your character to change simply because the **conversation is moving forward**?

This chapter shows you how to:

1. Change personality based on **message count**
2. Make characters act differently at different **times of day**
3. Trigger special **events** at certain milestones

Message Count Progression (Growing Friendships)

One of the easiest ways to add realism is to let the character "warm up" as the chat goes on.

```
var count = context.chat.message_count;

if (count < 5) {
context.character.personality += ", polite and formal";
context.character.scenario += " This feels like a cautious first meeting.";
} else if (count < 15) {
context.character.personality += ", becoming more casual";
context.character.scenario += " The atmosphere is loosening up.";
} else if (count < 30) {
context.character.personality += ", open and friendly";
context.character.scenario += " You've both settled into an easy rhythm.";
} else {
context.character.personality += ", deeply connected";
context.character.scenario += " The bond feels strong and genuine.";
}
```

Plain English:

- Early messages → polite stranger
- Midway → casual and relaxed
- Long chats → trust and deep connection

This is like a **relationship arc** unfolding as you keep talking.

Time-Based Changes (Day and Night Personality)

Scripts can also read the clock! That means you can change how your character acts at night vs. day.

```
var hour = new Date().getHours();

if (hour < 6 || hour > 22) {
context.character.personality += ", a bit sleepy";
context.character.scenario += " It's late at night, and everything feels quiet.";
} else {
context.character.personality += ", bright and energetic";
context.character.scenario += " It's daytime, the world is busy around you.";
}
```

Plain English:

- If it's past 10 PM or before 6 AM → character feels sleepy

- Otherwise → character feels awake and lively

This makes conversations feel grounded in a *living world*.

Event Triggers (Special Surprises)

You can create little "story beats" that happen at certain times in the chat.

```
if (context.chat.message_count === 10) {
context.character.personality += ", momentarily distracted";
context.character.scenario += " Suddenly, their phone rings with an unexpected call.";
}

if (context.chat.message_count === 25) {
context.character.personality += ", reactive to the environment";
context.character.scenario += " The weather suddenly changes around them.";
}
```

Plain English:

- At 10 messages: A phone rings (mini-event)
- At 25 messages: The weather shifts

This gives the illusion that the *story has beats* like a TV episode.

Keyword + Timing = Extra Flavor

You can also mix timing and keyword checks.

```
var last = context.chat.last_message.toLowerCase();

if (context.chat.message_count > 15 && last.indexOf(" secret ") !== -1) {
context.character.personality += ", mysterious and cautious";
context.character.scenario += " They whisper, as if revealing something hidden.";
}
```

Plain English:

- Only after 15+ messages...
- If the user mentions "secret"...
- The character reveals hidden knowledge

This feels like *unlocking lore* through deeper conversation.

Putting It All Together

Dynamic behaviors make your character:

- **Evolve over time** (message count)
- **Feel tied to the world** (day/night cycles)
- **Experience surprises** (events at milestones)
- **Reveal secrets naturally** (timed keyword gates)

Even if you never touch "advanced lorebooks," just adding **message count + time-based + event triggers** can make your bot feel much richer.

Key Takeaways from Chapter 6

- Use **message count** to simulate relationship growth
- Use **time of day** for realism (night vs. day moods)
- Sprinkle in **event triggers** for surprise moments
- Combine timing + keywords for "unlockable" secrets

Pro Tip: Don't overload your character with too many events at once. Just 2–3 well-placed beats can make a chat feel cinematic.

Chapter 7: Memory & Preferences (Making Bots "Remember")

Here's the truth: scripts don't actually *remember* things the way humans do. Every time the chat moves forward, the script starts fresh.

But! We can **fake memory** by writing details into the `scenario` (or sometimes `personality`). Since the model "reads" these fields before generating a reply, it will act like it remembered.

Think of it like jotting notes on a sticky pad:

- User: "My name is Alex."
- Script writes: "Remember: user's name is Alex" into the scenario.
- Now the bot "sees" that note in future turns.

Trick 1: Capturing Names

```
var last = context.chat.last_message.toLowerCase();

if (last.indexOf("my name is") !== -1) {
var match = context.chat.last_message.match(/my name is (\w+)/i);
if (match) {
context.character.scenario += " Remember: the user's name is " + match[1] + ".";
}
}
```

Plain English:

- If the user types "my name is ..." → capture the word after it
- Add a note to the scenario: "Remember: the user's name is Alex."
- Now the bot will act like it knows your name later

Trick 2: Storing Interests (Likes and Dislikes)

We can also detect hobbies, foods, or other favorites.

```
var last = context.chat.last_message.toLowerCase();
var likes = ["pizza", "movies", "music", "hiking"];
var dislikes = ["spiders", "loud noises", "crowds"];

for (var i = 0; i < likes.length; i++) {
if (last.indexOf(likes[i]) !== -1) {
context.character.personality += ", remembers the user likes " + likes[i];
context.character.scenario += " They bring up " + likes[i] + " as a friendly topic.";
}
}

for (var j = 0; j < dislikes.length; j++) {
if (last.indexOf(dislikes[j]) !== -1) {
context.character.personality += ", remembers the user dislikes " + dislikes[j];
context.character.scenario += " They avoid mentioning " + dislikes[j] + ".";
}
}
```

Plain English:

- If the user says they like pizza → the bot remembers and might mention it

- If they say they dislike spiders → the bot avoids that topic
- These get added into personality and scenario as notes

Trick 3: Memory by Repetition

Scripts can also add reminders over time:

context.character.personality += ", has a good memory for conversation details";
context.character.scenario += " They remember important things the user has shared.";

Plain English:
Even if you don't capture a name or hobby, you can add flavor text that says the bot "remembers." This nudges the AI to act consistent with earlier lines.

Trick 4: Hybrid Lore + Memory

You can combine lore with memory, so the bot responds differently based on what the user likes.

Example:

- If user says they like "stars," and then mentions "magic," the lore might be written with a positive spin:
    - "Magic feels harmonious, like a song from the stars."
- If user dislikes "darkness," the same lore shifts to caution:
    - "Magic can be dangerous, especially when tied to shadows."

Plain English:
This makes the world feel **tailored** to the user, like the character is really paying attention.

Quick Practice (Try It Yourself!)

1. Make the bot **remember a favorite color** if the user says "I like blue."
2. Make the bot **avoid scary topics** if the user says "I'm afraid of spiders."
3. Make the bot **store a pet's name** if the user says "My dog's name is Max."

(Hint: use the same pattern as the "my name is" example, but change the word.)

Key Takeaways from Chapter 7

- Scripts don't really "remember," but you can fake it with `scenario` notes
- Capture names, hobbies, likes, dislikes with simple keyword checks
- Personality/scenario additions guide the bot to act consistent
- Hybrid systems combine lore with preferences for personal flavor

---

Pro Tip: Don't overload memory with too many notes. A few well-placed reminders ("user likes pizza," "user's name is Alex") go a long way.

Chapter 8: Building a Simple Lorebook

Up until now, you've been working with **triggers**: single words → single responses. That's fine for small scripts, but what if you want your bot to *remember multiple pieces of world information*?

That's where a **lorebook** comes in.

A **lorebook** is just a collection of entries. Each entry is like a **mini fact**:

- Who someone is
- What a place looks like
- How a character reacts

Step 1: The Mini Entry (One Fact)

Here's the smallest possible "lorebook":

```
var lorebook = [
{ keywords: ["forest"], personality: ", at home in nature", scenario: "Tall trees sway in the breeze." }
];

var last = context.chat.last_message.toLowerCase();
var padded = " " + last + " ";

for (var i = 0; i < lorebook.length; i++) {
var entry = lorebook[i];
for (var j = 0; j < entry.keywords.length; j++) {
if (padded.indexOf(" " + entry.keywords[j] + " ") !== -1) {
context.character.personality += entry.personality;
context.character.scenario += entry.scenario;
```

```
break;
}
}
}
```

Plain English:

- `lorebook` is just a list (array) of entries
- Each entry has **keywords**, a **personality trait**, and a **scenario note**
- If the user says “forest,” the bot adds forest lore

Step 2: Multiple Entries

Now let’s add more facts.

```
var lorebook = [
{ keywords: ["forest"], personality: ", at home in nature", scenario: "Tall trees sway in the breeze." },
{ keywords: ["city"], personality: ", sharp and streetwise", scenario: "The streets buzz with activity." },
{ keywords: ["river"], personality: ", calm and reflective", scenario: "Water flows gently nearby." }
];
```

Plain English:
Now “forest,” “city,” and “river” each unlock their own lore.

Step 3: Priorities

What if multiple entries trigger at once? We don’t want everything to fire.
We add **priority numbers**. Higher priority wins.

```
var lorebook = [
{ keywords: ["godfather", "damien"], priority: 10, personality: ", a calculating and charismatic leader" },
{ keywords: ["mafia", "family"], priority: 5, personality: ", part of a powerful crime family" }
];

var activated = [ ];
for (var i = 0; i < lorebook.length; i++) {
var entry = lorebook[ i ];
for (var j = 0; j < entry.keywords.length; j++) {
if (padded.indexOf(" " + entry.keywords[ j ] + " ") !== -1) {
activated.push(entry);
break;
```

```
}
}
}

activated.sort(function(a, b) { return b.priority - a.priority; });

if (activated.length > 0) {
context.character.personality += activated[0].personality;
}
```

Plain English:

- If both “godfather” and “family” appear, “godfather” wins because it has higher priority

---

Step 4: Flat Style (Beginner-Friendly)

If arrays feel overwhelming, you can just write entries as `if` checks.

```
if (padded.indexOf(" forest ") !== -1) {
context.character.personality += ", at home in nature";
context.character.scenario += "Tall trees sway in the breeze.";
}

if (padded.indexOf(" city ") !== -1) {
context.character.personality += ", sharp and streetwise";
context.character.scenario += "The streets buzz with activity.";
}
```

Plain English:
This works the same as a lorebook — it’s just less organized. Fine for small scripts, but messy for big worlds.

Step 5: Expanding Lore

Lorebooks can include more than just traits and places. You can add:

- **Relationships** (e.g., “brother,” “mentor”)
- **Objects** (e.g., “sword,” “ring”)
- **Factions** (e.g., “mages guild,” “alchemists”)

Example:

```
var lorebook = [
{ keywords: ["mentor"], personality: ", wise and strict", scenario: "Their mentor watches closely." },
{ keywords: ["ring"], scenario: "A mysterious ring glints faintly." }
];
```

Plain English:
Anything you want the bot to "know about" can go into a lorebook.

Recap Table

Style	Pros	Cons	Best For
Flat if checks	Easy to read, no arrays	Gets messy fast	Beginners, small projects
Simple lorebook array	Organized, scalable	Slightly harder syntax	Medium projects
Lorebook w/ priorities	Resolves conflicts, neat	Needs sorting step	Complex projects
Expanded lorebook	Covers people, places, objects	More setup work	Large worldbuilding

---

Key Takeaways from Chapter 8

- A **lorebook** is just a structured list of entries
- Each entry = keywords + personality + scenario
- Use **priorities** if multiple entries might trigger
- Start with flat if checks, move to arrays as your world grows
- Lorebooks keep scripts clean and organized — essential for big projects

---

Pro Tip: Think of a lorebook like a **wiki inside your script**. Each entry is a "page" (a fact), and keywords are the links that lead to it.

Chapter 9: Best Practices & Performance

By now, you've seen scripts grow from **tiny triggers** to **dynamic lorebooks**. That's powerful — but it also means there's more room for mistakes. This chapter teaches you how to keep scripts:

- **Safe** (they don't crash)

- **Efficient** (they don't slow down)
- **Readable** (you can come back later and understand them)

Rule 1: Keep It Lightweight

Remember: scripts run **every single message**. If your script is too heavy, it can cause slowdowns or fail silently.

- **Good:** A few if checks, some loops under 50 items
- **Bad:** 1,000+ keyword checks in one block

Tip: Split giant lists into categories (greetings, emotions, lore). Process only what you need.

Rule 2: Loop Carefully

Loops are safe, but they can get out of hand.

```
for (var i = 0; i < list.length; i++) {
if (padded.indexOf(" " + list[ i ] + " ") !== -1) {
// do something
break; // stop once found!
}
}
```

Plain English: Always add `break;` when you only need one match. This prevents extra unnecessary checks.

Rule 3: Always Append, Never Overwrite

You should **add to personality/scenario**, not replace them.

Wrong:

```
context.character.personality = "Happy";
```

(Overwrites everything!)

Right:

```
context.character.personality += ", feeling happy";
```

Plain English: Think of `+=` like adding a note to a notebook instead of ripping out all the old pages.

Rule 4: Clamp Your Outputs

Each field has a safe limit before the AI might start ignoring or dropping text.

- **Safe limit:** ~600 characters per entry
- **Danger zone:** 2,000+ characters in one personality/scenario update

Keep your additions short and punchy. Example:

- Good: ", a little tired but still smiling."
- Bad: ", who begins to recall an entire 3-paragraph monologue about last summer..."

Rule 5: Test, Don't Assume

Not all features work in the sandbox (remember Chapter 3!). If you're unsure, **test it first**.

For example, instead of assuming `.includes` works, always check with `.indexOf` (the guaranteed safe method).

Rule 6: Use Debugging

You can peek at what's happening by printing to the console:

```
console.log("Message count:", context.chat.message_count);
console.log("Last message:", context.chat.last_message);
console.log("Personality so far:", context.character.personality);
```

Plain English: This is like turning on the lights backstage. You can see what's actually being passed into your script.

Rule 7: Handle Nothing Gracefully

Sometimes the user won't trigger *any* keywords. That's fine! Always make sure your script leaves a fallback:

```
if (noMatchFound) {
context.character.personality += ", neutral and calm";
}
```

Plain English: The character should still have *something* to work with — don't leave fields empty.

Rule 8: Avoid "Overstuffing"

It's tempting to load the script with every possible keyword. But overstuffing makes characters *robotic*.

Instead:

- Pick 5–10 meaningful triggers
- Expand only when you're sure it adds fun

Rule 9: Use Comments

Comments are notes for yourself inside the code. They don't affect the script.

```
// This block checks for greetings
if (padded.indexOf(" hello ") !== -1) {
context.character.scenario += "They greet warmly.";
}
```

Plain English: Always leave breadcrumbs so Future You remembers what each block does.

Rule 10: Plan for Growth

Think of scripts as gardens:

- Start with a few plants (basic triggers)
- See how they grow in real conversations
- Only then add new plants (lore, memory, events)

If you try to plant everything at once, it becomes a jungle you can't manage.

Key Takeaways from Chapter 9

- Keep scripts light and efficient
- Use loops carefully with `break;`
- Always **append, never overwrite**
- Clamp outputs (short, atomic sentences)
- Test features before relying on them
- Debug with `console.log` when stuck
- Plan small and grow gradually

Pro Tip: The best scripts aren't the biggest ones. They're the ones that feel invisible — just enough to make the character seem alive without drawing attention to the code behind it.

Chapter 10: Putting It All Together

We've learned the pieces one by one:

- How to match words (Chapter 4)
- How to build progressive triggers (Chapter 5)
- How to use time and message count for pacing (Chapter 6)
- How to fake memory with notes (Chapter 7)
- How to use structured lorebooks (Chapter 8)
- And how to keep things safe and efficient (Chapter 9)

Now, let's **combine them all** into one simple but powerful script.

Step 1: Always Start with Guards

Every script should start by making sure the fields exist.

```
// === CONTEXT GUARDS ===
context.character = context.character || {};
context.character.personality = context.character.personality || "";
context.character.scenario = context.character.scenario || "";
```

Plain English:

- If personality or scenario doesn't exist yet, create them as empty strings
- This prevents errors before we do anything else

Step 2: Prepare the Message

We want the last user message in lowercase and padded with spaces.

```
var last = String((context.chat && context.chat.last_message) || "");
var padded = " " + last.toLowerCase() + " ";
```

Plain English:

- `toLowerCase()` → makes it case-insensitive
- `" " + … + " "` → makes it safe to check whole words

Step 3: Add Keyword Reactions

We'll do greetings, emotions, and secrets.

```
// Greetings
var greetings = ["hello", "hi", "hey"];
for (var i = 0; i < greetings.length; i++) {
if (padded.indexOf(" " + greetings[i] + " ") !== -1) {
context.character.scenario += " They greet you warmly.";
context.character.personality += " Friendly and welcoming.";
break;
}
}

// Emotions
var emotions = ["happy", "sad", "angry", "excited"];
for (var j = 0; j < emotions.length; j++) {
if (padded.indexOf(" " + emotions[j] + " ") !== -1) {
context.character.scenario += " The user seems " + emotions[j] + ".";
break;
}
}

// Secrets
if (padded.indexOf(" secret ") !== -1) {
context.character.personality += " Becomes mysterious when secrets are mentioned.";
context.character.scenario += " The atmosphere shifts into secrecy.";
}
```

Plain English:

- If the user greets → the bot greets back
- If they express an emotion → the bot notices it
- If they mention a secret → the bot becomes mysterious

Step 4: Add Message Count Progression

We'll make the character grow friendlier the longer the chat goes.

```
var count = context.chat.message_count;

if (count < 5) {
context.character.personality += ", polite and cautious.";
} else if (count < 15) {
context.character.personality += ", warming up and more casual.";
} else if (count < 30) {
context.character.personality += ", open and relaxed.";
} else {
context.character.personality += ", deeply connected and trusting.";
}
```

Plain English:

- Short chat → polite
- Medium chat → casual
- Long chat → close friend

Step 5: Add Time-of-Day Flavor

Let's make night feel sleepy, day feel lively.

```
var hour = new Date().getHours();

if (hour < 6 || hour > 22) {
context.character.personality += ", a bit sleepy.";
context.character.scenario += " It's late at night, and everything feels quiet.";
} else {
context.character.personality += ", alert and energetic.";
context.character.scenario += " The world outside is lively.";
}
```

Plain English:

- Early morning / late night → sleepy
- Daytime → energetic

Step 6: Add Simple Memory (Name Capture)

We'll make the bot "remember" if the user says their name.

```
if (last.indexOf("my name is") !== -1) {
var match = context.chat.last_message.match(/my name is (\w+)/i);
if (match) {
context.character.scenario += " Remember: the user's name is " + match[1] + ".";
}
}
```

Plain English:

- If the user says "my name is ..." → store it in scenario
- Now the bot acts like it remembers your name

Step 7: Add a Mini Lorebook

Finally, let's add some simple world lore.

```
var lorebook = [
{ keywords: ["forest"], personality: ", deeply connected to nature", scenario: " They are surrounded by tall trees and rustling leaves." },
{ keywords: ["city"], personality: ", street-smart", scenario: " The bustling city streets surround them." }
];

for (var k = 0; k < lorebook.length; k++) {
var entry = lorebook[k];
for (var m = 0; m < entry.keywords.length; m++) {
if (padded.indexOf(" " + entry.keywords[m] + " ") !== -1) {
context.character.personality += entry.personality;
context.character.scenario += entry.scenario;
break;
}
}
}
```

Plain English:

- If the user mentions “forest” → add nature lore
- If the user mentions “city” → add city lore
- These change the “stage set” of the conversation

Step 8: Put It All Together

At the end of the script, the character will:

- React to greetings, emotions, and secrets
- Grow more comfortable with message count
- Act differently depending on the time of day
- Remember a name if the user says it
- Add lore if certain keywords appear

That’s a **complete, beginner-friendly script** that covers almost every trick we’ve learned so far.

Key Takeaways from Chapter 10

- Always start with **guards**
- Use **safe matching** (`toLowerCase` , `indexOf` , padding)
- Stack behaviors: keyword triggers + progression + time + memory + lore
- Keep entries **short and atomic**
- Build slowly and test often

Pro Tip: Don’t copy this script wholesale into your bot. Instead, use it as a **template**. Delete the parts you don’t need, keep the parts you do, and expand with your own creativity.

Chapter 11: Quick Reference Cheat Sheet

Congratulations! You’ve made it through the beginner guide.
This chapter is your **pocket survival kit** for writing scripts. Keep it handy — it’s everything you really need at a glance.

Safe Tools (Always Work)

- **Text**

- `toLowerCase()` → makes text lowercase
- `indexOf(" word ") !== -1` → check if a word is present
- `trim()` → removes spaces

- **Numbers & Math**
  - `+`, `-`, `*`, `/` → basic math
  - `Math.random()` → random number 0–1
  - `Math.floor()` → round down

- **Arrays**
  - `arr.length` → how many items
  - `arr.indexOf("thing")` → check if "thing" is in list
  - `for` loops → loop through items

- **Dates**
  - `new Date().getHours()` → current hour

- **Regex**
  - `/\bword\b/i.test(text)` → check whole word safely

- **Debugging**
  - `console.log("Message:", context.chat.last_message);`

Unsafe Tools (Never Work)

- `.map()`, `.filter()`, `.reduce()`, `.forEach()`
- Arrow functions `() => {}`
- Template strings `` `Hello ${name}` ``
- Spread operator `...`
- `async/await`, Promises
- Classes
- `try/catch` (errors can't be caught)
- `setTimeout`, `setInterval`, external calls (`fetch`)

Gray Area Tools (Avoid Them)

- `.includes()`
- `.repeat()`
- `.padStart()` / `.padEnd()`

They sometimes work, sometimes don't, depending on the host. Beginners: stick with `.indexOf`.

Common Patterns

Greeting Trigger

```
if (padded.indexOf(" hello ") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
}
```

Multiple Keywords

```
var words = ["hi", "hey", "hello"];
for (var i = 0; i < words.length; i++) {
if (padded.indexOf(" " + words[i] + " ") !== -1) {
// do something
break;
}
}
```

Message Count Progression

```
if (context.chat.message_count > 10) {
context.character.personality += ", more comfortable now.";
}
```

Time of Day Flavor

```
var hour = new Date().getHours();
if (hour < 6 || hour > 22) {
context.character.personality += ", sleepy.";
}
```

Name Capture

```
if (last.indexOf("my name is") !== -1) {
var match = context.chat.last_message.match(/my name is (\w+)/i);
```

```
if (match) context.character.scenario += " Remember: user's name is " + match[1] + ".";
}
```

Lorebook Entry

```
var lore = [
{ keywords: ["forest"], personality: ", loves nature", scenario: "They walk among tall trees." }
];
```

Best Practices Recap

- Always **append (`+=`)**, never overwrite
- Keep sentences **short and atomic**
- Loops: use `break;` to stop early
- Test features before relying on them
- Add **comments** to explain what code does
- Don't overload with giant word lists
- Don't write paragraphs into personality/scenario — keep it bite-sized

---

Quick Troubleshooting

- **"My script does nothing"** → You probably used an unsupported feature (e.g., `.includes`)
- **"It's repeating the same trait a lot"** → Add a check (`if (!context.character.personality.includes("trait"))`)
- **"It's too slow"** → Cut down loops or break earlier
- **"It forgot what I told it"** → Remember: fake memory by writing into `scenario`

The Golden Formula

If you ever get lost, here's the **minimum skeleton**:

```
context.character = context.character || {};
context.character.personality = context.character.personality || "";
context.character.scenario = context.character.scenario || "";

var last = String((context.chat && context.chat.last_message) || "");
var padded = " " + last.toLowerCase() + " ";
```

```
// Example reaction
if (padded.indexOf(" hello ") !== -1) {
context.character.scenario += "They greet you warmly.";
context.character.personality += "Friendly and welcoming.";
}
```

That's all you really need to start building. Everything else is just stacking more blocks on top.

Pro Tip: The best scripts aren't the fanciest. They're the ones that quietly nudge your character into feeling alive, without you ever noticing the machinery behind it.

Chapter 12: Weighted Lore & Probability

Up until now, your scripts have been **deterministic** — meaning: if the user types a word, the script *always* triggers the same response. That's great for consistency, but it can feel predictable.

What if sometimes the bot shares a story, but other times it stays quiet?
What if mentioning "magic" doesn't always flood the scene with spell lore?

That's where **weights and probability** come in.

Why Use Probability?

Humans aren't machines — we don't always react the same way every time. By adding probability, you can make responses feel:

- **Fresh** → The same word doesn't always trigger
- **Unpredictable** → Surprises keep the conversation alive
- **Natural** → Sometimes people mention something but don't elaborate

---

The Random Roll

The sandbox has a built-in dice roller:

```
Math.random()
```

This gives a number between **0 and 1** (like a percentage).

- `0.0` = 0%

- `0.5` = 50%
- `1.0` = 100%

So if you want something to happen 50% of the time:

```
if (Math.random() < 0.5) {
// do the thing
}
```

Adding Probability to Lore

Here's how we can add probability to a lore entry:

```
var last = context.chat.last_message.toLowerCase();

if (last.indexOf("magic") !== -1) {
if (Math.random() < 0.5) { // 50% chance
context.character.personality += ", remembers old magical teachings.";
context.character.scenario += " The air hums with faint magical energy.";
}
}
```

Plain English:

- If the user mentions "magic"...
- Roll the dice
- If the roll is under 0.5 (50% chance) → trigger the lore
- Otherwise → nothing happens (bot stays quiet)

Weighted Choices (Like a Roulette Wheel)

Probability doesn't have to be "yes or no." You can also make the bot **pick between multiple options**.

```
var options = [
{ chance: 0.6, text: "They talk about a magical library." },
{ chance: 0.3, text: "They recall a battle with a sorcerer." },
{ chance: 0.1, text: "They stay silent, eyes distant." }
];

var roll = Math.random();
var total = 0;
```

```
for (var i = 0; i < options.length; i++) {
total += options[i].chance;
if (roll < total) {
context.character.scenario += options[i].text;
break;
}
}
```

Plain English:

- Options are given **weights** (60%, 30%, 10%)
- Roll the dice
- Whichever slot the dice falls into → that's the chosen outcome
- So "magical library" happens most often, but sometimes you'll get the rarer paths

---

Realistic Example: Telling Secrets

```
if (last.indexOf("secret") !== -1) {
var roll = Math.random();
if (roll < 0.7) {
context.character.personality += ", reluctant but burdened with knowledge.";
context.character.scenario += " They hint at a secret but don't reveal it.";
} else {
context.character.personality += ", daring enough to share forbidden truths.";
context.character.scenario += " They whisper the real secret with trembling lips.";
}
}
```

Plain English:

- 70% of the time → the bot stays cagey
- 30% of the time → the bot spills the secret
- This feels human, because sometimes people hold back

Best Practices for Probability

- Use probability for **flavor**, not for everything
- Keep rare events **special** (don't hide key lore behind a 1% chance)
- Document your weights with comments so you remember why you picked them

- Don't chain too many random checks at once — randomness piles up and makes scripts unpredictable in bad ways

---

Quick Practice (Try It Yourself!)

1. Add a **50% chance** that the bot mentions the weather when the user says "outside."
2. Create a **weighted choice** where "forest" triggers either:
    - 60% → peaceful description
    - 30% → mysterious atmosphere
    - 10% → dangerous vibes

Key Takeaways from Chapter 12

- `Math.random()` gives you a 0–1 roll (your digital dice)
- Use `< number` checks for simple percentages
- Use **weighted choices** for more variety
- Add probability for flavor, surprise, and realism

Pro Tip: Think of probability like seasoning in cooking — a little makes things delicious, but too much can ruin the dish.

Chapter 13: Min/Max Message Gating (Unlocking Content Over Time)

So far, we've used **message count** for gradual shifts (polite → casual → trusting). But what if you want to **lock and unlock certain lore** depending on how long the conversation has been going?

That's what **min/max gating** is for. It's like setting a **window of opportunity**:

- *Before 15 messages → the secret is hidden*
- *Between 16–30 messages → the secret is revealed*

This creates natural pacing, like chapters in a story.

Why Use Gating?

- **Story beats** → certain reveals only happen once the bond deepens
- **Mystery** → early hints, later explanations
- **Progression** → the chat feels like it has "levels"

The Simple Formula

You can check message count with two conditions:

```
var count = context.chat.message_count;

if (count >= 5 && count <= 15) {
context.character.scenario += " They seem hesitant to share anything personal yet.";
}
```

Plain English:

- If message count is **between 5 and 15** → add this scene note
- Outside that range → nothing happens

Example: Secrets in Stages

Let's use gating to reveal a secret over time.

```
var count = context.chat.message_count;

if (count <= 15 && padded.indexOf(" secret ") !== -1) {
context.character.personality += ", cautious about their secrets.";
context.character.scenario += " They hint that there are things they cannot share yet.";
}

if (count >= 16 && count <= 30 && padded.indexOf(" secret ") !== -1) {
context.character.personality += ", finally ready to open up.";
context.character.scenario += " They whisper a deeper truth, as if trusting you more.";
}

if (count > 30 && padded.indexOf(" secret ") !== -1) {
context.character.personality += ", burdened by secrets too heavy to ignore.";
context.character.scenario += " They reveal everything, unable to hold it in any longer.";
}
```

Plain English:

- Early in the chat → they avoid the secret
- Midway → they share cautiously
- Later → they spill everything

Example: Event Unlock

You can also tie events to certain ranges:

```
if (count === 10) {
context.character.scenario += " A distant bell rings, marking a turning point in the conversation.";
}

if (count > 20 && count < 25) {
context.character.personality += ", feeling nostalgic.";
context.character.scenario += " They recall something from their childhood.";
}
```

Plain English:

- At exactly 10 messages → an event happens
- Between 20 and 25 messages → they enter a nostalgic mood

Best Practices for Gating

- Use **ranges** for flexibility (e.g., 15–30), not just single numbers
- Tie gates to **story pacing** (early, mid, late)
- Combine with **keywords** (like "secret") for more depth
- Don't make everything gated — the chat shouldn't feel like a checklist

---

Quick Practice (Try It Yourself!)

1. Make a character **stay guarded** before 10 messages, but **warm up** between 10–20
2. Add an **event** at exactly 25 messages where "a storm begins"
3. Make the bot **reveal a family story** only if message count is above 30

Key Takeaways from Chapter 13

- Use `>=` and `<=` to create message count **windows**
- Gating creates pacing and unlocks lore naturally
- Combine gating with **keywords** for deeper reveals
- Treat gating like **chapters** in a conversation — new arcs appear as the chat grows

---

Pro Tip: Think of min/max gating as *doors in a hallway*. Each door only opens after enough steps forward, revealing a new part of the story.

Chapter 14: Shifts & Conditional Layers

So far, our lore entries have been **flat**: one keyword → one response. But real life isn't flat. A forest feels different at **day** vs. **night**. A character acts differently when they're **calm** vs. **angry**.

That's where **shifts** (conditional layers) come in. They let one entry change flavor depending on extra context.

Step 1: Flat Entry (No Shift)

```
if (padded.indexOf(" forest ") !== -1) {
context.character.scenario += "The forest surrounds you.";
}
```

Plain English:
Anytime the user says "forest," we just add: *"The forest surrounds you."*
Always the same, no matter what else is happening.

Step 2: Adding a Shift (Day vs. Night)

```
if (padded.indexOf(" forest ") !== -1) {
if (padded.indexOf(" day ") !== -1) {
context.character.scenario += "The forest feels alive in the daylight.";
} else if (padded.indexOf(" night ") !== -1) {
context.character.scenario += "The forest feels eerie under the moonlight.";
} else {
context.character.scenario += "The forest surrounds you.";
}
}
```

Plain English:

- If “forest” + “day” → bright forest
- If “forest” + “night” → eerie forest
- Otherwise, default forest

Step 3: Lorebook with Shifts

We can store shifts **inside an entry**.

```
var lorebook = [
{
keywords: ["forest"],
scenario: "The forest surrounds you.",
shifts: [
{ keywords: ["day"], scenario: "The forest glows with sunlight." },
{ keywords: ["night"], scenario: "The forest grows dark and quiet." }
]
}
];
```

Processing looks like:

1. Match “forest.”
2. Add its base scenario.
3. Check if any **shift keywords** also match → add their text.

Step 4: Emotional Shifts

Shifts don’t have to be time-based — they can be **emotional layers**.

```
var lorebook = [
{
keywords: ["mentor"],
personality: ", wise and strict",
shifts: [
{ keywords: ["trust"], personality: ", softens when trusted" },
{ keywords: ["anger"], personality: ", harsh when angry" }
]
}
];
```

Plain English:

- “Mentor” = wise and strict
- If “trust” is present → add soft trait
- If “anger” is present → add harsh trait

Step 5: Probabilistic Shifts

You can add **random variety** into shifts.

```
if (padded.indexOf(" tavern ") !== -1) {
if (Math.random() < 0.5) {
context.character.scenario += "The tavern is rowdy tonight.";
} else {
context.character.scenario += "The tavern is quiet and dimly lit.";
}
}
```

Plain English:
Same keyword, two possible moods. Keeps the world fresh.

Step 6: Layered Shifts (Stacking)

Multiple conditions can layer together.

```
if (padded.indexOf(" forest ") !== -1) {
if (padded.indexOf(" night ") !== -1) {
context.character.scenario += "The forest is dark and silent.";
if (padded.indexOf(" wolves ") !== -1) {
context.character.scenario += "You hear wolves howling in the distance.";
}
}
}
```

Plain English:

- Forest + Night → dark forest
- Forest + Night + Wolves → adds a howling event

Recap Table

Type	Example	Use Case
Flat	"Forest surrounds you"	Always the same response
Time-based shift	"Day → bright" / "Night → eerie"	Environmental changes
Emotional shift	"Mentor + trust → softer"	Character reactions
Probabilistic shift	Tavern noisy vs. quiet	Variety / replayability
Layered shift	Night + Wolves = howling	Stacking depth

Key Takeaways from Chapter 14

- Shifts = **conditional flavors** that modify a base entry
- Use them for **time, mood, emotion, or randomness**
- Shifts make the world **react to context** instead of staying flat
- Layer shifts for **rich storytelling** without bloating your script

---

Pro Tip: Think of shifts like *lighting in a movie scene*. The set (forest) doesn't change, but the lighting (day, night, candle, storm) transforms the mood completely.

Chapter 15: Ordered Keywords (Sequential Logic)

So far, your triggers have been simple:

- "happy" → add cheerful trait
- "forest" → add forest scene

But what if you only want something to fire if **keywords appear in order**?

Example: "She teases you… then admits her feelings."

- If the user only says "tease," it shouldn't fire
- If they only say "feelings," it shouldn't fire
- But if "tease" comes **before** "feelings," it unlocks a special event

This is called **sequential logic.**

Step 1: Flat Keywords (No Order)

```
if (padded.indexOf(" tease ") !== -1) {
context.character.personality += ", playful and teasing.";
}
if (padded.indexOf(" feelings ") !== -1) {
context.character.scenario += "They hint about their deeper feelings.";
}
```

Plain English:

- If “tease” shows up → playful
- If “feelings” shows up → emotional hint
- Works fine, but doesn’t check *order*

Step 2: Multi-Keyword (Both Present, Any Order)

```
if (padded.indexOf(" tease ") !== -1 && padded.indexOf(" feelings ") !== -1) {
context.character.scenario += "Their teasing shifts into something heartfelt.";
}
```

Plain English:
This only triggers if both words appear, but it doesn’t care which one comes first.

Step 3: Sequential Logic (Order Matters)

```
var teaseIndex = padded.indexOf(" tease ");
var feelingsIndex = padded.indexOf(" feelings ");

if (teaseIndex !== -1 && feelingsIndex !== -1 && teaseIndex < feelingsIndex) {
context.character.scenario += "The teasing slowly turns into a confession of feelings.";
}
```

Plain English:

- Both words must appear
- “tease” must come **before** “feelings”
- Only then does the special scenario fire

Step 4: Story Example

User says:
"She confessed her feelings, almost teasing."
→ *Does not fire* (feelings came before tease)

User says:
"She kept teasing until she admitted her feelings."
→ *Fires* (tease came first)

Step 5: Expanded Sequences

You can extend this idea to longer chains.

```
var meetIndex = padded.indexOf(" meet ");
var fightIndex = padded.indexOf(" fight ");
var reconcileIndex = padded.indexOf(" reconcile ");

if (meetIndex !== -1 && fightIndex !== -1 && reconcileIndex !== -1) {
if (meetIndex < fightIndex && fightIndex < reconcileIndex) {
context.character.scenario += "They met, fought, and finally reconciled.";
}
}
```

Plain English:
The script only fires if the **full sequence** appears in order.

Recap Table

Type	Example	Order Matters?
Flat	"tease" → playful	No
Multi-keyword	"tease" + "feelings" → heartfelt	No
Sequential	"tease" → "feelings"	Yes
Long chain	meet → fight → reconcile	Yes

Key Takeaways from Chapter 15

- Sequential logic checks **position** as well as presence
- Lets you script **story beats** that unfold in order
- Great for arcs like: teasing → feelings, fight → reconcile, secret → betrayal
- Without order checks, scripts may fire in unintended contexts

---

Pro Tip: Think of sequential logic like *sheet music*. Notes (keywords) only make sense when they're played in the right order.

Chapter 16: Event Lore (Randomized Story Beats & Ambient Flavor)

Up until now, every script has been **user-driven**:

- The bot reacts to keywords
- The bot responds to message count

But what if sometimes *the world moves on its own*?

That's where **Event Lore** comes in. These are little **ambient events** — bells tolling, phones ringing, weather shifting — that fire on timers, random rolls, or story pacing. They add surprise and immersion, like background flavor in a movie scene.

Why Use Event Lore?

- Keeps the conversation world feeling *alive*
- Adds **surprise** — the user didn't "trigger" it, but it still happens
- Creates **beats** like in storytelling — little climaxes and turning points

---

Example 1: Timed Events

```
if (context.chat.message_count === 10) {
context.character.scenario += " A church bell rings in the distance, marking the tenth exchange.";
}

if (context.chat.message_count === 25) {
context.character.scenario += " A sudden breeze stirs, carrying whispers from nowhere.";
}
```

Plain English:

- At exactly 10 messages → bell sound
- At 25 messages → eerie breeze
- These happen *even if the user didn't mention anything*

Example 2: Random Events

```
if (Math.random() < 0.2) {
context.character.scenario += " A bird flutters past, wings scattering dust motes.";
}
```

Plain English:

- Every message, roll the dice
- 20% of the time → add a random environmental detail
- Feels like the world has background "ticks"

Example 3: Event Pools

```
var events = [
" A phone rings suddenly in the distance.",
" Thunder rumbles faintly overhead.",
" Someone knocks at the door unexpectedly."
];

if (Math.random() < 0.15) {
var pick = events[Math.floor(Math.random() * events.length)];
context.character.scenario += pick;
}
```

Plain English:

- 15% of the time → grab a random "ambient beat" from the pool
- This creates a rotation of surprises

Example 4: Event Lore + Keywords

```
if (padded.indexOf(" dream ") !== -1 && Math.random() < 0.3) {
context.character.scenario += " A dreamlike haze falls over the scene, blurring reality.";
}
```

Plain English:

- If “dream” is mentioned, sometimes (30% chance) the world itself becomes dreamlike

---

Best Practices for Event Lore

- Use sparingly — 1–2 ambient beats every 10–15 messages feels natural
- Keep events short and atmospheric
- Tie rare events to “big moments” (like a storm starting at message 50)
- Don’t spam random events every turn — it overwhelms the conversation

---

Quick Practice (Try It Yourself!)

1. Create a **10% chance per message** for a **mysterious shadow** to appear
2. Add an **event pool** with at least 3 “city noises” (sirens, honking, chatter)
3. Make an event where at **exactly 30 messages**, the character receives a **letter** that changes the tone of the conversation

---

Key Takeaways from Chapter 16

- **Event lore** creates surprises independent of user input
- Use **timed events** for predictable beats
- Use **random rolls** for ambient flavor
- Use **event pools** for variety
- Combine with **keywords** for rare, dramatic twists

Pro Tip: Think of event lore like a *movie soundtrack*. The characters may not control it, but it shapes how the scene feels.

Chapter 17: Simple Reaction Engines (Weighted Keyword Scores)

So far, your scripts have been **on/off switches**:

- If the user says “happy” → add cheerful personality
- If the user says “sad” → add somber personality

That’s good for basics, but what if you want your character to build up **stronger reactions** depending on *how many* related words show up?

That’s what a **reaction engine** does: instead of a single trigger, it **scores words** and reacts based on the total.

Why Use Weighted Scores?

- Captures **intensity** (a little teasing vs. a lot of teasing)
- Prevents **false positives** (one weak word doesn’t immediately flip a mood)
- Creates **gradual escalation** (small signals build into a bigger reaction)

Example 1: Scoring Touch Words

Let’s say we want the bot to react to physical contact, but only if enough signals appear.

```
var touchWords = ["touch", "hold", "grab", "caress"];
var score = 0;

for (var i=0; i<touchWords.length; i++) {
if (padded.indexOf(" " + touchWords[i] + " ") !== -1) {
score++;
}
}

if (score >= 2) {
context.character.personality += ", responsive to physical closeness.";
context.character.scenario += " Their body language shifts as the touch lingers.";
}
```

Plain English:

- Scan for “touch,” “hold,” “grab,” “caress.”
- Add 1 point for each match.
- If at least 2 are found → trigger the reaction.

Example 2: Weighted Keywords

Not all words are equal. “caress” might carry more weight than “touch.”

```
var reactions = [
{ word: "touch", weight: 1 },
{ word: "hold", weight: 2 },
{ word: "caress", weight: 3 }
];

var score = 0;
for (var i=0; i<reactions.length; i++) {
if (padded.indexOf(" " + reactions[i].word + " ") !== -1) {
score += reactions[i].weight;
}
}

if (score >= 3) {
context.character.personality += ", reacting strongly to intimacy.";
}
```

Plain English:

- “touch” = +1
- “hold” = +2
- “caress” = +3
- Add them up → the stronger the words, the faster the threshold is reached.

Example 3: Escalating Tiers

Once you have a score, you can make different levels of reaction.

```
if (score === 1) {
context.character.personality += ", slightly responsive.";
} else if (score === 2) {
context.character.personality += ", noticeably moved.";
} else if (score >= 3) {
context.character.personality += ", deeply affected.";
}
```

Plain English:

- 1 point → mild response
- 2 points → stronger
- 3+ points → intense

This feels much more natural than instant jumps.

Best Practices for Reaction Engines

- Use scores for **emotions, physical actions, or tone shifts**
- Assign weights carefully (not every word is equal)
- Keep thresholds low (2–3 points is usually enough)
- Don't make the math too complex — keep it simple and readable

---

Quick Practice (Try It Yourself!)

1. Make a reaction engine for **anger words** (angry, furious, rage).
    - 1 = mild annoyance, 2 = frustration, 3+ = full rage
2. Build a **flirtation engine** where playful words ("wink," "tease," "smirk") add up until the bot becomes overtly flirty
3. Create a **fear engine** where words like "dark," "scary," and "danger" escalate tension in the scenario

---

Key Takeaways from Chapter 17

- Reaction engines score **multiple inputs** instead of just flipping a switch
- Scores allow for **gradual escalation** and **intensity tiers**
- Weighted words add realism (some words count more)
- This is the first step toward **dynamic emotional engines**

Pro Tip: Reaction engines are like a thermometer — the more words you pile in, the hotter the mood gets.

Chapter 18: Performance & Sandbox Limits (How Far You Can Push Scripts)

As scripts grow, they can start failing *silently*. That's the sandbox at work. It's a safe, limited environment with strict rules. Knowing these rules saves hours of guessing.

Why Limits Exist

- **Safety** — prevents crashes and exploits
- **Speed** — scripts run on *every message*
- **Consistency** — everyone gets the same predictable behavior

---

Limit 1: JavaScript Version (ES5 Only)

Your script runs on an older JavaScript (ES5).

Works: `for` loops, `if/else`, `indexOf`, `toLowerCase`, string concatenation (`+`), basic math, `Math.random()`.
Doesn't work: arrow functions `() => {}`, template strings using backticks, `async/await`, classes, `.map()`, `.filter()`, `.reduce()`, `.forEach()`.
Rule of thumb: if it looks "modern JS," assume it's not supported here.

Limit 2: String Size

Safe per addition: about a few **hundred characters** (aim for ~**< 600**).
Risky: dropping in paragraphs (2,000+ chars) at once.
Hard ceiling per turn (all text combined): about **~27,000 characters**.
Implication: keep additions small and punchy so they aren't truncated or ignored.

Limit 3: Loops

Loops are fine when lean.
Safe: **< 300** checks.

Risky: **~1,000+** iterations or deep nesting.
Tip: exit early whenever possible using `break`.

Limit 4: Execution Time

Scripts get only a few milliseconds. If you do too much, the sandbox won't throw an error — it simply stops mid-run.
Implication: when "nothing happens," it's often *performance*, not logic.

Limit 5: Memory & State

No persistent memory between turns.
To "remember," write notes into `context.character.scenario` or (sparingly) `context.character.personality`.
Global variables won't persist.

Limit 6: Error Handling

No `try/catch`. If something is undefined, the script can die quietly.
Prevent that with guards at the top of every script:

```
context.character = context.character || {};
context.character.personality = context.character.personality || "";
context.character.scenario = context.character.scenario || "";
```

Limit 7: Randomness

`Math.random()` works and is great for flavor.
Don't hide critical plot points behind randomness — users might miss them.

Best Practices for Staying Safe

- Keep strings short and atomic (one idea per sentence).
- Cap loops and use `break` to stop early.
- Prefer `indexOf` over unsupported helpers (no `.includes()` in ES5).

- Always add guards before doing anything else.
- Build small, test often, then expand.

Quick Practice (Spot the Limits!)

1) Why will this often fail in the sandbox?

```
if (context.chat.last_message.includes("hello")) { /* ... */ }
```

Answer: `.includes()` isn't guaranteed in ES5; use `indexOf` with padding instead, like `padded.indexOf(" hello ") !== -1`.

2) What's risky here?

```
for (var i = 0; i < 1000; i++) { /* checks */ }
```

Answer: ~1,000 iterations can exceed safe execution time. Reduce the list, split it, or exit early with `break`.

3) Why is this dangerous?

```
context.character.scenario += "They write a whole novel chapter all at once ... (very long)";
```

Answer: Large additions risk truncation or being ignored and eat into the ~27k limit. Keep each addition short.

Key Takeaways from Chapter 18

- You are in **ES5** — no modern JS, no `.includes`, no arrow functions.
- Keep outputs short; safe per-addition target is **< ~600 chars**.
- Keep loops lean (**< 300** checks) and break early.
- No persistent memory; write "notes" into `scenario` to fake it.
- No `try/catch`; rely on **guards** to prevent undefined crashes.
- If a script "does nothing," suspect **performance**, not just logic.

Chapter 19: Definitional vs. Relational vs. Event Lore

Not all lore entries serve the same purpose. Some define **what things are**, some explain **how they connect**, and some move the story with **what happens.**

Think of it like writing a play:

- **Definitional lore** = the stage and cast list.
- **Relational lore** = how the cast feels about each other.
- **Event lore** = the script that makes things happen.

Type 1: Definitional Lore

What it is:

- Baseline facts about people, places, or objects.
- Things that don't change often.

Examples:

- "The Godfather is calculating and charismatic."
- "The forest is filled with tall pines."

Example code:

```
if (padded.indexOf(" godfather ") !== -1) {
context.character.personality += ", calculating and charismatic.\n\n";
context.character.scenario += "He sits in a lavish study.\n\n";
}
```

Plain English:

Definitional lore tells us *what exists.*

Type 2: Relational Lore

What it is:

- How characters, groups, or places connect.
- Explains bonds, rivalries, trust, or loyalty.

Examples:

- "Damien is loyal to family above all else."
- "The mage guild distrusts the alchemists."

Example code:

```
if (padded.indexOf(" family ") !== -1) {
context.character.personality += ", loyal to family above all.\n\n";
}
```

Plain English:
Relational lore tells us *how things relate.*

Type 3: Event Lore

What it is:

- Story beats that happen at a specific moment.
- Can be tied to time, message count, or triggers.

Examples:

- "At 20 messages, the phone rings."
- "When secrets are mentioned, they whisper about the Sundering."

Example code:

```
if (context.chat.message_count === 20) {
context.character.scenario += "A phone rings suddenly, breaking the silence.\n\n";
}
```

Plain English:
Event lore tells us *when things happen.*

Quick Comparison

Lore Type	Purpose	Example	Code Shape
Definitional	Establish facts	"Godfather is calculating"	Keyword → baseline trait
Relational	Define connections	"Mage guild distrusts alchemists"	Keyword → bond/rivalry
Event	Move story forward	"At 20 messages, a phone rings"	Message count / trigger

---

How They Work Together (Story Snippet)

User says: "Tell me about the Godfather and his family."

- **Definitional lore fires:**
  Godfather = calculating leader.
- **Relational lore fires:**
  Loyal to family above all.
- **Event lore adds (at msg 20):**
  A phone rings during the meeting.

Combined, this paints a rich scene: *A calculating Godfather, loyal family ties, and a sudden event interrupting.*

Key Takeaways from Chapter 19

- **Definitional lore** = what exists.
- **Relational lore** = how it connects.
- **Event lore** = when things happen.
- Together, they form a **world Bible** for your script.

Chapter 20: Dynamic Lore Systems (Mixing & Evolving Entries Over Time)

Up until now, your lore entries have been **static**:

- If a word shows up → add this personality/scene.
- If message count hits 10 → fire this event.

That's good, but static lore can feel predictable. A **dynamic lore system** lets entries:

- Change depending on *when* they fire.
- Build on each other in stages.
- Unlock or replace previous notes with new ones.

Think of it like a TV show: characters, relationships, and events **evolve** as the episodes go on.

Strategy 1: Progressive Lore Entries

Have the same keyword trigger **different effects** at different stages.

Example code:

```
var count = context.chat.message_count;

if (padded.indexOf(" forest ") !== -1) {
if (count < 10) {
context.character.scenario += " The forest feels calm and welcoming.\n\n";
} else if (count < 20) {
context.character.scenario += " The forest begins to feel mysterious, shadows lengthening.\n\n";
} else {
context.character.scenario += " The forest feels dangerous now, with unseen creatures watching.\n\n";
}
}
```

Plain English:

- Early: peaceful forest.
- Midway: mysterious forest.
- Later: dangerous forest.

Strategy 2: Evolving Traits

Replace or upgrade personality notes as time goes on.

Example code:

```
if (padded.indexOf(" trust ") !== -1) {
if (count < 15) {
context.character.personality += ", cautious about trust.\n\n";
} else {
context.character.personality += ", openly trusting now.\n\n";
}
}
```

Plain English:

The same keyword ("trust") means **different things** early vs. later in the chat.

Strategy 3: Unlock Chains

Lore can **unlock other lore.**

Example code:

```
if (padded.indexOf(" secret ") !== -1) {
context.character.scenario += " They hint at something hidden.\n\n";
context.character.personality += ", a keeper of secrets.\n\n";

// Unlock related lore
if (count > 20) {
context.character.scenario += " They finally share a secret about the Sundering.\n\n";
}
}
```

Plain English:

- Mentioning “secret” creates a new trait.
- If enough time has passed, it unlocks the *next layer* of lore.

Strategy 4: Lore Fusion

Two lore entries can combine into something new.

Example code:

```
if (padded.indexOf(" magic ") !== -1 && padded.indexOf(" forest ") !== -1) {
context.character.scenario += " The forest is alive with strange magical energy.\n\n";
}
```

Plain English:

- Normally, “magic” and “forest” have separate notes.
- If both appear → they fuse into a **unique blended event.**

Strategy 5: Probability-Based Lore Variants

Keep lore fresh by adding randomness.

Example code:

```
if (padded.indexOf(" dragon ") !== -1) {
if (Math.random() < 0.5) {
context.character.scenario += " A dragon roars in the distance.\n\n";
} else {
context.character.scenario += " A dragon flies overhead, wings blotting out the sun.\n\n";
```

```
}
}
```

Plain English:

- Mentioning “dragon” doesn’t always give the same response.
- Sometimes you get a roar, sometimes a sighting.

Best Practices for Dynamic Lore

- Use **message count** to stage lore progression.
- Upgrade traits instead of just stacking new ones.
- Unlock new entries gradually for pacing.
- Fuse multiple keywords for surprising combos.
- Add probability for variety.
- Don’t make every entry dynamic — keep some static for stability.

Quick Practice (Try It Yourself!)

1. Make a “river” lore entry that starts calm, then grows wild after 20 messages.
2. Create a “friendship” entry that changes from “cautious” to “trusting” after 15 messages.
3. Write a fusion entry where “fire” + “forest” creates a wildfire scene.

Key Takeaways from Chapter 20

- Dynamic lore **evolves over time** instead of staying flat.
- Use message count for pacing.
- Traits can **shift, unlock, or fuse** with others.
- Randomness keeps entries fresh.
- This turns static worlds into **living stories.**

Chapter 21: Adaptive Reaction Engines (Polarity, Negation & Composite Moods)

So far, our reaction engines have been about **adding points** until a threshold is hit. That's useful, but real emotions aren't one-directional. People can:

- Feel positive **or** negative about a topic (**polarity**).
- Cancel out reactions if something is denied (**negation**).
- Hold **mixed feelings** (like bittersweet emotions).

This is where adaptive engines shine: they calculate *direction* and *blend*, not just "on/off."

Polarity (Positive vs. Negative)

Instead of just counting, we give words **+ or – values.**

Example code:

```
var polarityWords = [
{ word: "love", score: +2 },
{ word: "like", score: +1 },
{ word: "hate", score: -2 },
{ word: "dislike", score: -1 }
];

var polarity = 0;
for (var i=0; i<polarityWords.length; i++) {
if (padded.indexOf(" " + polarityWords[i].word + " ") !== -1) {
polarity += polarityWords[i].score;
}
}

if (polarity > 0) {
context.character.personality += ", warm and affectionate.\n\n";
} else if (polarity < 0) {
context.character.personality += ", cold and distant.\n\n";
}
```

Plain English:

- Positive words push polarity up.
- Negative words push it down.
- Result = affectionate OR distant.

Negation (Canceling Out)

We also need to handle when a user says the **opposite**:

- "I'm *not* happy."
- "I *don't* like that."

We can scan for negation words before we score.

Example code:

```
var negation = (padded.indexOf(" not ") !== -1 || padded.indexOf(" don't ") !== -1);

if (padded.indexOf(" happy ") !== -1) {
if (negation) {
context.character.personality += ", notes the user isn't actually happy.\n\n";
} else {
context.character.personality += ", mirrors the user's happiness.\n\n";
}
}
```

Plain English:

- If "happy" is present → check if negation words appear nearby.
- If yes → treat it as *opposite*.
- If no → treat normally.

Composite Moods (Blends)

Sometimes two emotional signals combine into a **mixed state.**

Example code:

```
var happy = padded.indexOf(" happy ") !== -1;
var sad = padded.indexOf(" sad ") !== -1;

if (happy && sad) {
context.character.personality += ", sensing a bittersweet mix of joy and sadness.\n\n";
} else if (happy) {
context.character.personality += ", uplifted by joy.\n\n";
} else if (sad) {
context.character.personality += ", touched by sorrow.\n\n";
}
```

Plain English:

- If both "happy" and "sad" show up → treat as **bittersweet** instead of ignoring one.

Advanced Example: Weighted Polarity + Negation

Example code:

```
var polarityWords = [
{ word: "love", score: +2 },
{ word: "like", score: +1 },
{ word: "hate", score: -2 },
{ word: "angry", score: -1 }
];

var polarity = 0;
for (var i=0; i<polarityWords.length; i++) {
var w = polarityWords[i];
if (padded.indexOf(" " + w.word + " ") !== -1) {
var neg = (padded.indexOf(" not " + w.word) !== -1 || padded.indexOf(" don't " + w.word) !== -1);
polarity += (neg ? -w.score : w.score);
}
}

if (polarity > 1) {
context.character.personality += ", affectionate and engaged.\n\n";
} else if (polarity < -1) {
context.character.personality += ", hostile and dismissive.\n\n";
} else {
context.character.personality += ", neutral but observant.\n\n";
}
```

Plain English:

- Words push polarity positive or negative.
- Negation flips the meaning.
- End result = warm, hostile, or neutral depending on balance.

Best Practices for Adaptive Engines

- Use polarity when you want *direction* (love vs hate).
- Use negation so the bot doesn't misread "not happy" as "happy."
- Use composite moods for realism (bittersweet, conflicted).
- Don't overload with giant wordlists — start small (3–5 words each).

Quick Practice (Try It Yourself!)

1. Create a polarity engine for **trust vs. doubt.**
    - "trust" = +2, "doubt" = -2.
2. Add negation so "don't trust" = -2 instead of +2.
3. Build a composite mood where **"fear" + "hope"** = "anxious but determined."

Key Takeaways from Chapter 21

- Adaptive engines track **direction**, not just intensity.
- **Polarity** lets emotions swing positive or negative.
- **Negation** prevents misreads.
- **Composite moods** allow mixed feelings.
- Together, they make bots feel more human.

Chapter 22: Hybrid Emotional States (Blending Multiple Engines)

So far, we've built:

- **Simple engines** (counting words = reaction).
- **Adaptive engines** (positive vs negative, negation, composite moods).

But people don't just feel *one* thing at once.
You can be **nervous AND excited.**
You can be **angry AND affectionate.**

This is where **hybrid emotional states** come in: combining multiple engines to calculate a *blend*.

Why Use Hybrids?

- More realistic — characters act like people, not switches.
- Adds tension and nuance — "bittersweet," "playful but shy," etc.

- Lets you script **conflicting emotions** instead of forcing a single tone.

Strategy 1: Multiple Scores

Run two engines at the same time, then combine the results.

Example code:

```
// Excitement Engine
var exciteWords = ["excited", "thrilled", "can't wait"];
var excite = 0;
for (var i=0; i<exciteWords.length; i++) {
if (padded.indexOf(exciteWords[i]) !== -1) excite++;
}

// Fear Engine
var fearWords = ["scared", "afraid", "nervous"];
var fear = 0;
for (var j=0; j<fearWords.length; j++) {
if (padded.indexOf(fearWords[j]) !== -1) fear++;
}

// Blend
if (excite > 0 && fear > 0) {
context.character.personality += ", excited but nervous.\n\n";
context.character.scenario += " Their smile is wide, but their hands tremble.\n\n";
}
```

Plain English:

- If “excited” words and “fear” words both show up → bot enters a hybrid “nervous-excited” state.

Strategy 2: Weighted Blends

Not all emotions are equal. Maybe “fear” should outweigh “excitement.”

Example code:

```
var state = (excite * 1) + (fear * 2);

if (state >= 3 && fear > 0 && excite > 0) {
context.character.personality += ", anxious but determined.\n\n";
```

```
}
```

Plain English:

- Excitement counts less, fear counts more.
- Final mood is “anxious but determined.”

Strategy 3: Triangular States

You can blend **three engines** for even more realism.

Example code:

```
var love = padded.indexOf(" love ") !== -1 ? 1 : 0;
var jealous = padded.indexOf(" jealous ") !== -1 ? 1 : 0;
var anger = padded.indexOf(" angry ") !== -1 ? 1 : 0;

if (love && jealous) {
context.character.personality += ", affectionate but jealous.\n\n";
}
if (love && anger) {
context.character.personality += ", passionate but short-tempered.\n\n";
}
if (jealous && anger) {
context.character.personality += ", bitter and defensive.\n\n";
}
```

Plain English:

- Any **pair** of emotions creates a hybrid.
- You don’t need every possible combo — just the ones that matter to your story.

Strategy 4: Default + Hybrid Override

Sometimes you want one reaction engine to run normally... but then override it if hybrids are detected.

Example code:

```
if (excite > 0) {
context.character.personality += ", clearly excited.\n\n";
}
if (fear > 0) {
```

```
context.character.personality += ", visibly nervous.\n\n";
}

// Hybrid override
if (excite > 0 && fear > 0) {
context.character.personality += ", a jittery mix of excitement and nerves.\n\n";
}
```

Plain English:

- Base states always add something.
- If both exist, add a **hybrid note on top**.
- This creates layered complexity.

Best Practices for Hybrids

- Start with just 2–3 emotional axes (don't overload).
- Use hybrids for **contrast** (joy + fear, love + jealousy).
- Let one state **dominate** if needed (fear outweighs joy).
- Don't try to cover every possible combo — just the meaningful ones.

Quick Practice (Try It Yourself!)

1. Create a hybrid for **happy + tired** = "content but yawning."
2. Make **angry + sad** = "heartbroken rage."
3. Try adding a third axis: **love + fear + trust** — what kind of hybrid would that make?

Key Takeaways from Chapter 22

- Hybrid states let characters feel **multiple emotions at once.**
- Run multiple engines side by side.
- Use weighting to balance which emotion dominates.
- Hybrids add realism, tension, and nuance.

Pro Tip: Hybrid states are like music chords — one note alone is simple, but blending two or three creates richness and emotion.

Chapter 23: Error Guards & Sandbox Tricks (Keeping Scripts Safe and Stable)

If you've ever had a script suddenly stop working for no clear reason, you've probably hit a **sandbox limitation.** The problem is, the sandbox won't tell you what went wrong — it just fails silently.

The solution: **error guards** and **safe coding habits.**

What's an Error Guard?

An **error guard** is a little snippet of code at the start of your script that makes sure the sandbox won't crash if something is missing.

The "golden guard" looks like this:

```
// === CONTEXT GUARDS ===
context.character = context.character || {};
context.character.personality = context.character.personality || "";
context.character.scenario = context.character.scenario || "";
```

Plain English:

- If `context.character` doesn't exist → create it.
- If personality/scenario don't exist → make them empty strings.
- This prevents "undefined" errors from breaking the script.

Guarding Loops

Loops can crash scripts if they run too long. Add **limits** and **breaks.**

```
for (var i=0; i<keywords.length && i<100; i++) {
if (padded.indexOf(" " + keywords[i] + " ") !== -1) {
// do something
break; // stop once found
}
}
```

Plain English:

- Cap loops so they don't spin forever.
- Exit early when you've already found what you need.

Guarding Against Overwrites

Never replace personality or scenario. Always **append (`+=`)**.

Wrong:
context.character.personality = "angry";
(Deletes everything else!)

Right:
context.character.personality += ", now angrier than before.";

Plain English: Always add, never erase.

Sandbox Tricks

Here are a few survival hacks:

1. **No modern JavaScript.**
    - Doesn't work: arrow functions, `.map()`, `.includes()`, template strings, `async/await`.
    - Safe: `for` loops, `indexOf`, `+` string concatenation.
2. **Keep strings short.**
    - Stay under a few hundred characters per update.
    - Giant paragraphs risk being cut off or ignored.
3. **Randomness is fine.**
    - `Math.random()` works.
    - Use it for probability and event lore, but not for critical story beats.
4. **No persistent memory.**
    - Scripts reset every turn.
    - If you need "memory," write notes into `scenario` (see Chapter 7).
5. **Fail gracefully.**

- If a check doesn't match anything, just leave personality/scenario unchanged.
- Don't try to force output every turn.

Debugging Tip

You can use `console.log` to peek at what's happening:

```
console.log("Message count:", context.chat.message_count);
console.log("Last message:", context.chat.last_message);
```

Plain English: Logs let you see what the sandbox *thinks* is going on — super useful for troubleshooting.

Best Practices Recap

- Always start with **context guards.**
- Always lowercase and pad user input.
- Cap loops and add `break;` .
- Append strings instead of overwriting.
- Keep outputs small.
- Use debugging with `console.log` .
- Don't assume modern JavaScript works — it doesn't.

Quick Practice (Try It Yourself!)

1. Add context guards to this broken snippet:

```
context.character.personality += " cheerful";
```

2. Fix this unsafe loop:

```
for (var i=0; i<1000; i++) { ... }
```

3. Rewrite this dangerous overwrite safely:

```
context.character.scenario = "dark room";
```

Key Takeaways from Chapter 23

- Guards prevent crashes from undefined fields.
- Loops and strings must be **kept lean.**
- Always append, never overwrite.
- Scripts reset every turn — use scenario for "memory."
- Sandbox = ES5 only, no modern JS.

Pro Tip: Think of error guards as *seatbelts.* You hope you never need them, but when things go wrong, they keep your script from flying off the road.

Chapter 24: The Everything Lorebook (Modular Framework for People, Places, Traits & Events)

By now, you've seen how lore entries can define people, places, relationships, and events. But when scripts start getting big, it's easy to get lost.

The **Everything Lorebook** is a way to **organize lore into categories** so you can keep it clear and expandable.

Think of it like a filing cabinet:

- One drawer for **people.**
- One for **places.**
- One for **traits.**
- One for **events.**

The Core Structure

Here's the skeleton of an "Everything Lorebook":

```
var lorebook = {
people: [
{ keywords: ["damien", "godfather"], personality: ", calculating leader", scenario: "He sits in a lavish study." },
{ keywords: ["sophia"], personality: ", fiery and ambitious", scenario: "She moves with restless energy." }
],
places: [
```

{ keywords: ["forest"], scenario: "Tall pines surround the clearing.", personality: ", grounded in nature" },
{ keywords: ["city"], scenario: "The streets bustle with life.", personality: ", sharp and streetwise" }
],
traits: [
{ keywords: ["trust"], personality: ", cautious about trust" },
{ keywords: ["anger"], personality: ", prone to flashes of temper" }
],
events: [
{ trigger: "count==10", scenario: "A church bell tolls in the distance." },
{ trigger: "count>20", scenario: "A storm begins brewing overhead." }
]
};

Plain English:

- `people` → who the characters are.
- `places` → where things happen.
- `traits` → personality flags and behavior layers.
- `events` → timed or triggered story beats.

Processing the Lorebook

We loop through each category and check for matches.

// Process people
for (var i=0; i<lorebook.people.length; i++) {
var entry = lorebook.people[i];
for (var j=0; j<entry.keywords.length; j++) {
if (padded.indexOf(" " + entry.keywords[j] + " ") !== -1) {
context.character.personality += entry.personality || "";
context.character.scenario += entry.scenario || "";
break;
}
}
}

You'd do the same for `places`, `traits`, and `events` (with small tweaks).

Event Handling

Events are a little different: they don't rely on keywords, but on conditions.

```
var count = context.chat.message_count;

for (var i=0; i<lorebook.events.length; i++) {
var entry = lorebook.events[ i ];

if (entry.trigger === "count==10" && count === 10) {
context.character.scenario += entry.scenario + "\n\n";
}

if (entry.trigger === "count>20" && count > 20) {
context.character.scenario += entry.scenario + "\n\n";
}
}
```

Plain English:

- If the trigger condition is true → event fires.

Why Modular Lorebooks Are Powerful

- **Organization:** Keeps your entries neat and grouped.
- **Scalability:** Easy to expand — just add to the right category.
- **Flexibility:** You can apply different rules per category (e.g., probability for events, shifts for traits).
- **Reusability:** You can lift one category out (like "places") and use it in another project.

Expansion: Layers Within Categories

You can also make each category support **shifts, weights, and gates.**

Example:

```
{
keywords: ["magic"],
personality: ", wise in magic",
scenario: "The air hums with energy.",
shifts: [
{ keywords: ["stars"], scenario: "Magic glimmers like starlight." },
{ keywords: ["shadows"], scenario: "Magic feels heavy and dark." }
],
probability: 0.5,
```

minCount: 10
}

Plain English:

- Base: “magic” → wise in magic.
- Shifts: “stars” → light flavor, “shadows” → dark flavor.
- Probability: 50% chance to trigger.
- minCount: only works after 10+ messages.

Best Practices

- Separate lore into **people, places, traits, events.**
- Use categories for readability and scaling.
- Add layers (shifts, weights, probability, gates) only where needed.
- Don’t try to cram *everything* into one mega-entry — split it.

Quick Practice (Try It Yourself!)

1. Add a new **person** entry for “mentor” who is wise but strict.
2. Add a **place** entry for “desert” with shifting moods for “day” vs “night.”
3. Add a **trait** entry for “curiosity” that only unlocks after 15 messages.
4. Add an **event** entry that fires at message 30 — “an unexpected guest arrives.”

Key Takeaways from Chapter 24

- The **Everything Lorebook** is a modular way to organize big scripts.
- Categories = people, places, traits, events.
- Processing each category keeps things clean and scalable.
- Entries can support shifts, probability, and gating.
- This turns chaotic scripts into **structured world engines.**

Pro Tip: Think of the Everything Lorebook like a *world wiki inside your bot*. Each entry is a page, and the categories are your table of contents.

Chapter 25: Advanced Layered Lore (Mixing Modular Systems Together)

At this point you've seen:

- **Definitional lore** (facts about people, places, objects).
- **Relational lore** (connections, bonds, rivalries).
- **Event lore** (beats that fire on timing or triggers).
- **Dynamic lore** (entries that evolve with time or context).
- **The Everything Lorebook** (modular organization for people, places, traits, events).

Chapter 25 takes it further: showing how to **combine these systems** into one **layered engine**.

Why Layer Lore?

- Keeps worlds **organized** but **alive.**
- Lets one keyword trigger **multiple categories** at once.
- Supports **growth over time** without spaghetti code.

Think of it like an orchestra:
Definitional = instruments, Relational = harmonies, Event = percussion beats, Dynamic = changes in tempo.

Step 1: Multi-Category Entries

Sometimes a single keyword belongs in multiple drawers. For example, "forest" is both a place and an emotional tone.

```
var lorebook = {
places: [
{ keywords: ["forest"], scenario: "Tall trees sway in the wind." }
],
traits: [
{ keywords: ["forest"], personality: ", grounded and calm" }
]
};
```

Plain English:

Mentioning “forest” expands both scene and personality at once.

Step 2: Stacking Layers

Entries can fire **in order**: definitional first, then relational, then events.

```
if (padded.indexOf(" mentor ") !== -1) {
context.character.personality += ", wise and strict";
}

if (padded.indexOf(" mentor ") !== -1 && padded.indexOf(" trust ") !== -1) {
context.character.personality += ", softens when trusted";
}

if (count === 20 && padded.indexOf(" mentor ") !== -1) {
context.character.scenario += " The mentor shares a secret at this moment.";
}
```

Plain English:

- Baseline = wise mentor.
- Relation = softer if trust is mentioned.
- Event = unlocks at message 20.

Step 3: Probability + Layers

Combine randomness with layers for replayability.

```
if (padded.indexOf(" tavern ") !== -1) {
if (Math.random() < 0.5) {
context.character.scenario += " The tavern is loud and rowdy.";
} else {
context.character.scenario += " The tavern is quiet, a hushed corner of town.";
}
}
```

Plain English:

“Tavern” always fires, but its mood shifts randomly. Next time, the same keyword feels fresh.

Step 4: Modular + Dynamic Expansion

Everything Lorebook categories can each have **shifts, weights, and gates.**

```
var lorebook = {
traits: [
{
keywords: ["courage"],
personality: ", brave but uncertain",
shifts: [
{ keywords: ["fear"], personality: ", courage tested against fear" }
],
minCount: 10
}
]
};
```

Plain English:

- Base trait = courage.
- If "fear" is also present → courage changes flavor.
- Only activates after 10 messages.

Step 5: Fusion Across Categories

Two categories can merge into something new.

```
if (padded.indexOf(" magic ") !== -1 && padded.indexOf(" city ") !== -1) {
context.character.scenario += " The city hums with magical energy, streetlamps glowing with arcane fire.";
}
```

Plain English:
Magic + city fuse into a special hybrid entry.

Best Practices

- Start modular (people, places, traits, events).
- Add **layers** (shifts, gates, probability) only where meaningful.
- Let categories **cross-pollinate** for richer worlds.

- Don’t overload — keep layers atomic and short.

Quick Practice (Try It Yourself!)

1. Add a **place** entry for “desert” with day/night shifts.
2. Add a **trait** “jealousy” that only fires after 20 messages.
3. Fuse **love + jealousy** into “possessive affection.”

Key Takeaways from Chapter 25

- Advanced lore = **layers working together.**
- Definitional + relational + event + dynamic entries all coexist.
- Modular categories keep things clean.
- Shifts, probability, and gating add flavor.
- Fusion creates unique story beats.

Pro Tip: Think of layered lore like *stacking transparent sheets*. Each sheet adds detail, but together they form the full picture of a living world.

Glossary of Key Terms

Append (`+=`) → Adding text onto the end of an existing field, instead of replacing it. Example: `personality += ", happy"` adds without deleting what was already there.
Sandbox (Boundary) → The controlled environment where scripts run. It enforces limits (older JavaScript/ES5, short strings, lean loops).
Context Guards → A short snippet at the start of a script that ensures `personality` and `scenario` exist, preventing crashes.
Dynamic Lore → Lore that changes depending on time, message count, probability, or other triggers.
Event Lore → Story beats or ambient details that happen at certain counts, randomly, or independently of user input.
Everything Lorebook → A modular structure for organizing lore entries into categories like people, places, traits, and events.
Gating → Unlocking traits, events, or lore only when message count passes certain thresholds (min/max).
Hybrid Emotional States → When two or more emotions blend into a mixed mood (e.g., excited + scared = nervous excitement).

indexOf → The safe ES5 method to check if a string contains a word. Example: `padded.indexOf(" happy ") !== -1`.

Keywords → Words that trigger specific lore entries or reactions.

Lorebook → A collection of entries that add personality/scene details based on triggers.

Negation → Detecting when a word is canceled by "not" or "don't." Example: "not happy" ≠ happy.

Padded input → Adding spaces before and after text (`" " + text + " "`) so word checks are safe (avoids matching "hat" inside "that").

Personality → The part of the character definition that describes *who they are.* Scripts can append to it during conversation.

Polarity → Positive/negative scoring in reaction engines (e.g., love = +2, hate = -2).

Probability → Adding randomness with `Math.random()`, making some responses happen only sometimes.

Reaction Engine → A script that scores multiple words (and sometimes polarity/negation) to determine emotional states, rather than a simple on/off trigger.

Scenario → The part of the character definition that describes *what's happening around them.* Scripts can append events, places, or lore here.

Sequential Logic → Checking if two or more words appear in a specific order (e.g., "tease" → "feelings").

Shifts → Variants of a lore entry that change based on secondary keywords (e.g., "magic" with "stars" = light magic; "magic" with "shadows" = dark magic).

Appendix A: Common Code Patterns

> These snippets assume you've already set up:
>
> ```
> var last = context.chat.last_message.toLowerCase();
> var padded = " " + last + " ";
> var count = context.chat.message_count;
> ```

Greeting Trigger

```
if (padded.indexOf(" hello ") !== -1) {
context.character.scenario += "They greet you warmly.\n\n";
context.character.personality += "Friendly and welcoming.\n\n";
}
```

Message Count Gating

```
if (count >= 10 && count <= 20) {
context.character.personality += ", opening up more.\n\n";
}
```

Random Event

```
if (Math.random() < 0.2) {
```

```
context.character.scenario += "A bird flutters past.\n\n";
}
```

Reaction Engine (Anger)

```
var angerWords = ["angry", "mad", "furious"];
var score = 0;
for (var i = 0; i < angerWords.length; i++) {
if (padded.indexOf(" " + angerWords[i] + " ") !== -1) score++;
}
if (score >= 2) {
context.character.personality += ", visibly angry.\n\n";
}
```

Sequential Logic (Order Matters)

```
var teaseIndex = padded.indexOf(" tease ");
var feelingsIndex = padded.indexOf(" feelings ");
if (teaseIndex !== -1 && feelingsIndex !== -1 && teaseIndex <
feelingsIndex) {
context.character.scenario += "The teasing turns into a confession of
feelings.\n\n";
}
```

Appendix B: Sandbox Rules Cheat Sheet

- Allowed: `for` loops, `if/else`, `indexOf`, `toLowerCase`, string concatenation (`+`), basic math, `Math.random()`.
- Not Allowed (ES5 sandbox): `.map()`, `.filter()`, `.reduce()`, `.forEach()`, arrow functions `() => {}`, template strings using backticks, `async/await`, classes, `try/catch`.
- Safe loop size: under 300 checks (keep it lean; break early when possible).
- Safe string size per addition: under ~600 characters (short, atomic sentences).
- Rough hard ceiling per turn (all text combined): ~27,000 characters.
- No persistent memory — fake memory by writing notes into `scenario`.
- Fail gracefully — if nothing matches, it's okay to do nothing this turn.

Appendix C: Copy-Paste Templates

Error Guard Starter

```
context.character = context.character || {};
context.character.personality = context.character.personality || "";
context.character.scenario = context.character.scenario || "";
```

Everything Lorebook Starter

```
var lorebook = {
people: [],
places: [],
traits: [],
events: []
};
```

Weighted Choice Function

```
function weightedChoice(options) {
var roll = Math.random(), total = 0;
for (var i = 0; i < options.length; i++) {
total += options[i].chance;
if (roll < total) return options[i].text;
}
return "";
}
```

Appendix D: Practice Challenges

1. Make a lore entry for "ocean" that shifts between calm (day) and stormy (night).
2. Build a simple polarity engine for trust vs. doubt.
3. Add an event that fires at 15 messages: "a knock at the door."
4. Write a hybrid mood for happy + tired = "content but yawning."

> Bonus: Add a 10% chance per message for a tiny ambient event (e.g., "distant footsteps") and see how often it feels right vs. too frequent.

Index

(kept exactly as you had — alphabetized and clean, no fenced code)